

The big picture start here

You already know **linear regression** (fit a line to predict a number) and **logistic regression** (fit a line, squash it to a probability). Every algorithm in this sheet answers one question: “what if a single straight line is not enough?”

Decision Tree

a flowchart of yes/no rules

Random Forest

many trees vote

Boosting

trees fix each other's mistakes

SVM

widest-street boundary

K-Means

find groups, no labels

Time Series

forecast with time order

Words you must know (5 min)

- **Supervised:** we have the answer column (price, churn).
- **Unsupervised:** no answer column, find structure (K-Means).
- **Overfitting:** model memorises training data, fails on new data. **Underfitting:** too simple to learn the pattern.
- **Bias** = too simple, always wrong in the same way. **Variance** = too sensitive, changes a lot with new data.
- **Hyperparameter:** a setting you choose before training (tree depth, learning rate). Tuned with cross-validation.
- **Ensemble:** combine many weak models into one strong model.

Two ways to ensemble

	Bagging	Boosting
Models built	In parallel, independently	One after another
Each model sees	A random sample of the data	The mistakes of the previous one
Fixes	High variance (overfitting)	High bias (underfitting)
Example	Random Forest	AdaBoost, XGBoost, CatBoost

Analogy: bagging = ask 100 people independently and take the majority. Boosting = one student keeps re-studying only the questions they got wrong.

1. Decision Tree classification & regression

Gist: a game of 20 Questions. The tree asks the best yes/no question about one feature, splits the data, and repeats until the groups are (almost) pure. Prediction = follow the questions down to a leaf.

How it picks the best question

- It wants groups that are as **pure** as possible (mostly one class).
- **Gini** $1 - \sum p^2$ (0 = pure, 0.5 = fully mixed for 2 classes).
- **Entropy** $-\sum p \cdot \log_2 p$.
- **Information gain** = impurity before split – weighted impurity after. Pick the split with the biggest gain.
- Regression trees use variance / MSE instead and predict the **average** of the leaf.

Worked example. A node has 10 loan applicants: 6 repaid, 4 defaulted. Gini = $1 - (0.6^2 + 0.4^2) = 0.48$. Split on “income > 50k”: left has 5 repaid, 0 default (Gini 0), right has 1 repaid, 4 default (Gini 0.32). Weighted = $0.5 \times 0 + 0.5 \times 0.32 = 0.16$. Gain = $0.48 - 0.16 = 0.32$, a great split.

Strengths / weaknesses

- + Easy to explain to business (draw the tree). No scaling needed. Handles numbers and categories, captures non-linear patterns and interactions.
- – **Overfits badly** if grown fully (one leaf per row). Unstable: small data change = different tree.

Controls (hyperparameters)

`max_depth` `min_samples_leaf` `min_samples_split` `ccp_alpha` (pruning). Smaller depth = simpler = less overfit.

`sklearn` `DecisionTreeClassifier(max_depth=4)`

Interview: “Why do trees overfit?” They can keep splitting until every training row has its own leaf. Fix: limit depth, prune, or use an ensemble. “Do trees need scaling?” No, splits depend on order, not size.

2. Random Forest bagging of trees

Gist: one tree is a moody expert; a forest is a committee. Grow many trees, each on a **random sample of rows** (bootstrap) using a **random subset of columns** at every split. Classification = **majority vote**, regression = **average**.

- **Why it works:** each tree makes different errors; averaging cancels them. Same bias as one tree, much lower variance.
- **OOB score:** each tree skips ~37% of rows; those rows give a free validation score (`oob_score=True`).
- **Feature importance:** how much each column reduced impurity across all trees. Good for a first ranking, but biased toward high-cardinality columns.
- **Key hyperparameters:** `n_estimators` (more trees = more stable, never overfits more), `max_depth`, `max_features`, `min_samples_leaf`.
- + Strong default, little tuning, robust to outliers, handles missing-ish noise.
- – Less interpretable, bigger and slower than one tree, poor at extrapolating beyond the training range.

Example. Predict customer churn with 100 trees. 63 trees say “churn”, 37 say “stay” → prediction = churn, and `predict_proba` = 0.63.

Interview: “Random Forest vs Decision Tree?” Forest = many de-correlated trees averaged, so lower variance and better accuracy, but less explainable.

3. Boosting — the core idea sequential learning

Gist: build a small, weak model. See where it is wrong. Build the next model to focus on those errors. Repeat 100–1000 times and add everything up. Each tree is tiny (a “stump” or depth 3–6), but the team is very accurate.

Step 1
weak model, rough guess

Step 2
measure the errors

Step 3
next model targets the errors

Step 4
add it in, small step
(learning rate)

Repeat
until errors stop shrinking

Numeric feel (house price, in lakhs). Start with the mean: 50. A house is really 60, so the residual is +10. Tree 2 learns to predict that +10. With learning rate 0.5, new prediction = $50 + 0.5 \times 10 = 55$. Next residual is +5, tree 3 adds $0.5 \times 5 = 2.5$, giving 57.5, creeping toward 60.

- **Learning rate** (shrinkage): small steps = need more trees but generalise better.
- **n_estimators**: number of rounds. Use **early stopping** on a validation set.
- Trade-off: `low learning_rate` + `many trees` = best, slowest.

3a. AdaBoost re-weights rows

Gist: after each weak model (usually a depth-1 stump), **raise the weight of rows it got wrong** so the next stump pays more attention to them. Final answer = **weighted vote**; better stumps get a louder voice.

Example. 100 rows, each weight 1%. Stump 1 misclassifies 10 rows → those rows now carry far more weight, stump 2 is forced to get them right, and so on.

- + Simple, few parameters (`n_estimators`, `learning_rate`).
- – Sensitive to noisy labels and outliers (it keeps chasing them).

`sklearn` `AdaBoostClassifier(n_estimators=100)`

3b. Gradient Boosting fits the residuals

Gist: instead of re-weighting rows, each new tree is trained to predict the **errors (residuals) of the current model**, i.e. it follows the gradient of the loss function downhill. This is exactly the house-price example on the left.

- Works with any differentiable loss (MSE, log-loss, MAE).
- Parents of XGBoost, LightGBM and CatBoost.
- – Slow on big data, many knobs, can overfit if too many rounds.

`sklearn` `GradientBoostingClassifier()`

3c. XGBoost eXtreme Gradient Boosting

Gist: gradient boosting engineered to be fast and hard to overfit. The go-to for tabular data and Kaggle.

- **Regularisation** (L1/L2 on leaf weights) and tree pruning built in.
- **Handles missing values**: learns the best default direction for NaN.
- **Parallel** and optimised; supports early stopping.
- Needs **numeric input**: encode categories first (one-hot / label).

Key params: `n_estimators` `learning_rate` `max_depth` `subsample` `colsample_bytree` `reg_lambda`

`code` `XGBClassifier(n_estimators=300, learning_rate=0.05, max_depth=4)`

3d. CatBoost Categorical Boosting

Gist: gradient boosting that **understands text/category columns directly**, so you skip one-hot encoding. Great when you have city, product, plan-type, etc.

- **Native categorical handling** using target statistics computed safely (no data leakage).
- **Ordered boosting** reduces overfitting on small datasets.
- Strong results with **default settings**; symmetric trees make prediction fast.
- – Training can be slower than LightGBM; heavier install.

Key params: `iterations` `learning_rate` `depth` `cat_features=[...]`

`code` `CatBoostClassifier(cat_features=["city"])`

Boosting family at a glance

Algorithm	What it does differently	Best when	Watch out
AdaBoost	Re-weights misclassified rows, depth-1 stumps	Small, clean data; quick baseline	Noisy labels, outliers
Gradient Boosting	Each tree fits previous residuals	Medium data, need good accuracy	Slow, easy to over-train
XGBoost	Regularised, fast, handles NaN	Default choice for tabular data	Must encode categories
LightGBM	Grows leaf-wise, histogram based, very fast	Very large data (millions of rows)	Can overfit small data
CatBoost	Native categories, ordered boosting	Many categorical columns, small data	Slower training

Interview: “Bagging vs boosting?” Bagging trains independent models on random samples to cut variance; boosting trains dependent models sequentially to cut bias. “Why is XGBoost popular?” Speed, built-in regularisation, missing-value handling, and top accuracy on tabular data. “How do you stop boosting overfitting?” Lower learning rate, limit depth, subsample, early stopping.

4. Support Vector Machine (SVM) classification (also regression)

Gist: logistic regression finds a line that separates two classes. SVM finds the **best** line: the one with the **widest empty street** (margin) between the classes. The few points touching the edge of the street are the **support vectors**; only they decide the boundary.

Three ideas

- **Maximum margin:** a wider street means the model is less likely to misclassify new points.
- **Soft margin – parameter C:** real data overlaps. **Small C** = allow some mistakes, wide street (simpler). **Large C** = punish every mistake, narrow street (risk of overfitting).
- **Kernel trick:** when no straight line works, lift the data into a higher dimension where a line does. The kernel does this without computing the new coordinates.

Kernel intuition. 1-D data: class A at $x = -1, 0, 1$ and class B at $-4, 4$. No single cut separates them. Add a second feature x^2 : A becomes $(-1,1), (0,0), (1,1)$ and B becomes $(-4,16), (4,16)$. Now the horizontal line $x^2 = 8$ splits them perfectly.

Practical notes

Kernel	Use for
linear	Many features, roughly linear (text)
rbf	Default for curved boundaries; tune <code>gamma</code>
poly	Polynomial-shaped boundaries

- **gamma** (rbf): high = each point has a tiny reach, wiggly boundary, overfits. Low = smooth.
- **Must scale features** (StandardScaler); SVM is distance based.
- + Strong with many features and small/medium data; effective in high dimensions.
- – Slow on very large data; no direct probabilities (`probability=True` adds them); hard to explain.

`sklearn SVC(kernel="rbf", C=1, gamma="scale")`

Interview: "What are support vectors?" The training points closest to the boundary; delete any other point and the model does not change. "Logistic regression vs SVM?" Logistic minimises log-loss and gives probabilities; SVM maximises the margin and can use kernels for non-linear boundaries.

5. K-Means Clustering unsupervised

Gist: no target column. You ask: "split my customers into K groups of similar people." K-Means puts K centres on the map and keeps nudging them until the groups settle.

1. Choose K

e.g. 3 groups

2. Drop K centres

random (k-means++)

3. Assign

each point to nearest centre

4. Move

centre = mean of its points

5. Repeat

until nothing moves

Tiny example. Values 1, 2, 3 and 10, 11, 12 with $K = 2$. Start centres at 1 and 12. Round 1: {1,2,3} and {10,11,12}. New centres = 2 and 11. Re-assigning changes nothing, so it stops. Clusters found, with no labels given.

Business use. Mall customers by *annual income* and *spending score*: "high income, low spend", "young big spenders", "budget shoppers" → one marketing plan per segment.

Choosing K

- **Elbow method:** plot **inertia** (sum of squared distances to the centre) against K; pick the bend where improvement flattens.
- **Silhouette score** (–1 to 1): closer to 1 = tight, well-separated clusters.

Rules of thumb

- **Always scale** features (income in lakhs would drown out age).
- Sensitive to outliers and start points (use `n_init`). Assumes round, similar-sized clusters. Numeric data only.

`sklearn KMeans(n_clusters=4, n_init=10)`

Interview: "Is K-Means supervised?" No. "How do you pick K?" Elbow plus silhouette plus business sense (how many segments can the team act on?). "K-Means vs KNN?" K-Means groups unlabeled data; KNN classifies using labeled neighbours.

Which algorithm when? quick chooser

Algorithm	Task	Scaling needed?	Non-linear?	Explainable?	Reach for it when...
Linear / Logistic	Reg / Class	Helps	No	High	You need a simple, defensible baseline
Decision Tree	Both	No	Yes	High	Rules must be shown to business
Random Forest	Both	No	Yes	Medium	Strong accuracy with minimal tuning
XGBoost / CatBoost	Both	No	Yes	Low–Med	Best accuracy on tabular data
SVM	Both	Yes	With kernel	Low	Small data, many features
K-Means	Clustering	Yes	–	Medium	No labels; customer segmentation

6. Time Series Forecasting Exponential Smoothing · ARIMA · SARIMA

Gist: ordinary regression assumes rows are independent. In a time series the **order matters**: today depends on yesterday. We forecast the future from the past of the *same* series (monthly sales, daily orders, weekly footfall).

Four building blocks of any series

- **Trend:** long-term rise or fall.
- **Seasonality:** repeating pattern (December spike every year).
- **Cyclic / noise:** irregular ups and downs, randomness.
- **Stationary:** mean, variance and pattern do not drift over time. ARIMA needs this. Check with the **ADF test** ($p < 0.05$ = stationary). Fix by **differencing**: subtract yesterday from today.

Differencing. Sales 100, 110, 121, 133 (trending up) → differences 10, 11, 12 (steady). The trend is gone, so the series is easier to model.

Rules for time series

- **Never shuffle or random-split.** Train on the past, test on the most recent period (e.g. hold out the last 12 months).
- **Metrics:** MAE, RMSE, **MAPE** (% error, easy to explain).
- **Baseline first:** “same as last month” or “same as last year”. If your model cannot beat it, it is not useful.
- **ACF / PACF plots** help choose ARIMA orders.

6a. Exponential Smoothing (ETS / Holt-Winters)

Gist: a weighted average of the past where **recent values count more** and old values fade away exponentially. Fast, simple, great baseline.

- **Simple (SES):** level only, no trend/season. $F(t+1) = \alpha \cdot \text{Actual}(t) + (1-\alpha) \cdot F(t)$
- **Holt:** level + **trend**.
- **Holt-Winters:** level + trend + **seasonality** (additive if swings are constant, multiplicative if swings grow with the level).
- **α near 1** = trust the latest value; **near 0** = smooth heavily.

Example. Last forecast for daily orders was 100, actual came in at 110, $\alpha = 0.5$. New forecast = $0.5 \times 110 + 0.5 \times 100 = 105$. With $\alpha = 0.9$ it would be 109: it chases the latest value.

```
code ExponentialSmoothing(y, trend="add",
seasonal="add", seasonal_periods=12).fit()
```

6b. ARIMA(p, d, q)

Gist: predict tomorrow from **yesterday's values** and **yesterday's forecast errors**, after removing the trend.

- **AR(p) – Autoregressive:** uses the last p values. It is linear regression of the series on its own past. **PACF** cut-off suggests p .
- **I(d) – Integrated:** number of times we difference to make it stationary (usually 0, 1 or 2).
- **MA(q) – Moving Average:** uses the last q forecast errors. **ACF** cut-off suggests q .

Example. ARIMA(1,1,0) on monthly sales: difference once, then predict this month's *change* from last month's change. If last change was +10 and the AR coefficient is 0.6, expect about +6 this month.

```
code ARIMA(y, order=(1,1,1)).fit()
```

Handles trend, **not seasonality**. Needs a single series with no big structural breaks.

6c. SARIMA(p,d,q)(P,D,Q)m

Gist: ARIMA plus a **seasonal** copy of the same ideas that looks back one full season.

- **(p,d,q):** the ordinary ARIMA part.
- **(P,D,Q):** same AR / differencing / MA but applied at seasonal lags.
- **m:** season length — 12 for monthly data, 7 for daily-with-weekly-pattern, 4 for quarterly.
- **D = 1** means subtract the value from one season ago (this December – last December).

Example. Monthly ice-cream sales peak every summer. SARIMA(1,1,1)(1,1,1)₁₂ models the trend with (1,1,1) and the yearly summer cycle with (1,1,1) at lag 12. add exog = **SARIMAX** (weather, promotions).

```
code SARIMAX(y, order=(1,1,1), seasonal_order=
(1,1,1,12)).fit()
```

Pick a time-series model

Your series has...	Use
Flat, no trend or season	Simple Exp. Smoothing
Trend only	Holt / ARIMA
Trend + seasonality	Holt-Winters / SARIMA
External drivers (price, promo)	SARIMAX, or XGBoost on lag features

Interview: “ARIMA vs SARIMA?” SARIMA adds seasonal terms. “Why difference?” To make the series stationary. “Why not random split?” It leaks the future into training.

60-minute teaching plan

5 min	Vocabulary: overfitting, bias/variance, bagging vs boosting
10 min	Decision tree + Gini worked example
8 min	Random Forest (vote, OOB, importance)
12 min	AdaBoost → Gradient Boosting → XGBoost → CatBoost
8 min	SVM: margin, C, kernel trick
7 min	K-Means: steps, elbow, segmentation
10 min	Time series: ETS, ARIMA, SARIMA